

YANFLÈ

Documentation d'architecture technique

YANFLÈ fait passer la vidéosurveillance d'un rôle de témoin — qui ne sert qu'à constater un acte après coup — à un rôle de **vigie** : un système qui comprend ce qu'il observe sur un réseau de caméras, pour aider à **prévenir un danger avant qu'il ne se produise**, plutôt que de simplement le filmer. Ce document décrit l'architecture technique du MVP qui met en œuvre une première partie de cette vision, exécuté localement sur un serveur équipé GPU.

Table des matières

1	Vue d'ensemble	1
2	La chaîne d'analyse	3
3	Reconnaissance faciale à deux mémoires	4
4	Narration factuelle (sans IA générative)	4
5	Modèle de données	4
6	Stockage	5
7	Sécurité	5
8	Limites connues et pistes	5

1 Vue d'ensemble

YANFLÈ transforme des caméras ordinaires (téléphones ou PC sur le même réseau) en un réseau de vidéosurveillance intelligent. Chaque image capturée traverse une **chaîne de 9 modèles de détection** plus un module de reconnaissance faciale, et le tout est résumé par un narrateur factuel. Le système est organisé autour de **cinq interfaces web**, chacune pour un usage précis, et d'un **serveur central** qui fait tourner l'intelligence artificielle.

Ce document par rapport au dossier de pitch

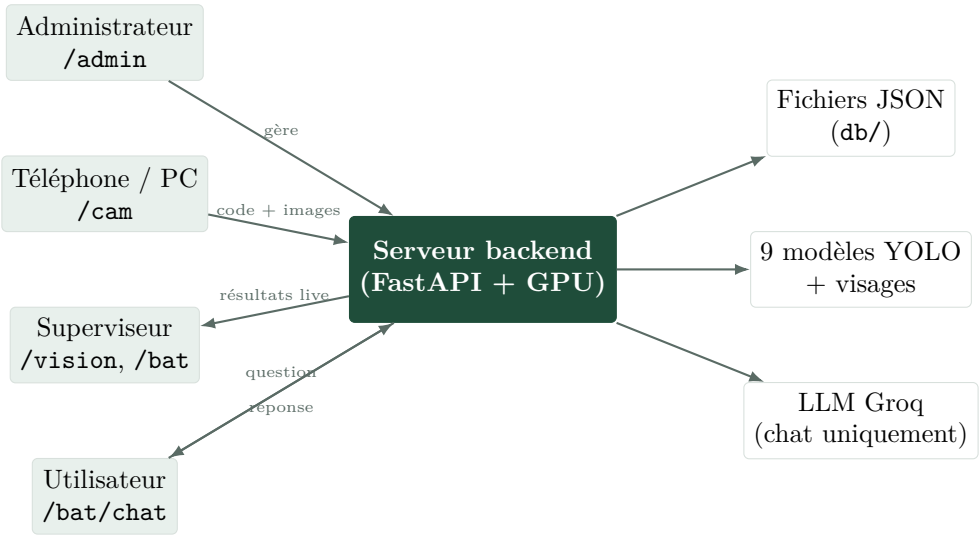
Le dossier de présentation initial (*YANFLÈ — Orange Summer Challenge 2026*) posait une **vision en quatre principes**, à un stade où *"la conception, l'entraînement du modèle et les tests rest[ai]ent entièrement à faire"*. Ce document décrit l'architecture du **MVP** qui en a été tiré depuis : une preuve de concept fonctionnelle, mais partielle, et qui s'écarte par endroits — volontairement — de la vision initiale. Le tableau ci-dessous fait le point, principe par principe.

Principe du pitch	État dans ce MVP
1. Observer le comportement, pas l'identité	Écart assumé. Le MVP ajoute une reconnaissance faciale (Le Phrysiomiste, §3) avec gestion d'autorisations par lieu — l'inverse du principe initial. Ce choix a été fait en cours de route pour permettre la détection de présence non autorisée, plus concrète à démontrer qu'une simple analyse de posture.
2. Reconnaître les signaux qui précèdent un acte	Partiellement couvert. Le Secouriste repère une chute déjà survenue (règle géométrique sur le squelette). Aucun modèle ne détecte encore un comportement <i>précurseur</i> (rôdage prolongé, posture d'agression avant le geste) — c'est la brique la plus proche de la promesse initiale du projet, et elle reste à construire.
3. Alerter un humain, ne jamais décider seul	Respecté. Tous les résultats (<code>/vision</code> , <code>/bat</code>) sont présentés à un superviseur humain ; le système ne déclenche aucune action automatique, il signale.
4. Adapter la réponse à la gravité (appel vocal automatisé)	Non implémenté. La narration et les alertes visuelles (dangers, présences non autorisées) existent, mais aucune escalade par appel téléphonique automatisé n'a été développée.

Les cinq interfaces

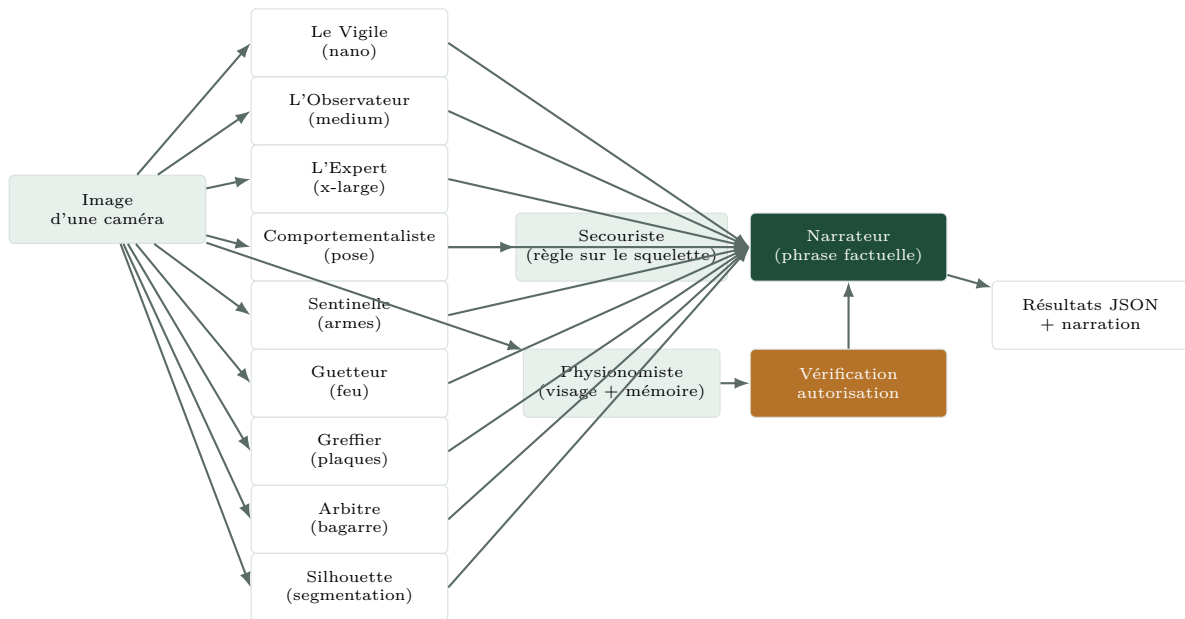
Route	Rôle
<code>/admin</code>	Créer des bâtiments , des points (portail, bureau, ...) et des caméras (chacune avec un code unique généré automatiquement).
<code>/cam</code>	Interface qu'un téléphone ou PC ouvre pour devenir une caméra du réseau : il entre le code, puis démarre/arrête l'envoi de son flux.
<code>/vision</code>	Vue de supervision globale : toutes les caméras de tous les bâtiments, en direct, groupées par bâtiment.
<code>/bat</code>	Vue de supervision scopée à un seul bâtiment (via son propre code) : caméras en direct, narration unique combinée, et gestion des personnes/autorisations.
<code>/bat/chat</code>	Assistant conversationnel : poser des questions en langage naturel sur l'état réel d'un bâtiment.

Schéma général



2 La chaîne d'analyse

Chaque image envoyée par une caméra (via /cam) ou par la page de démonstration passe par la même chaîne, exécutée entièrement sur le serveur.



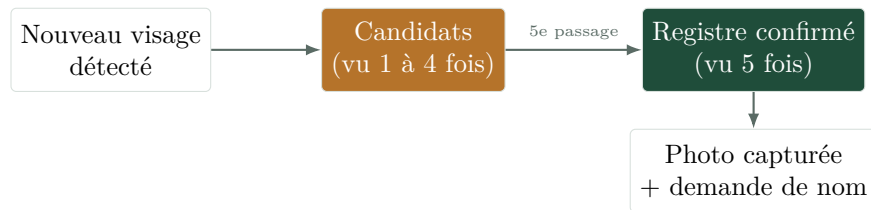
Détail des 9 modèles + 2 modules dérivés

Rôle	Ce qu'il fait
Le Vigile	Détection rapide (modèle nano), premier coup d'œil.
L'Observateur	Détection intermédiaire (medium), plus fiable.
L'Expert	Détection la plus précise (x-large), référence pour la narration.
Le Comportementaliste	Squelette (17 points clés) + suivi de chaque personne dans le temps (<code>track_id</code>).
La Sentinelle	Détection d'armes/objets dangereux (modèle communautaire, seuil de confiance relevé).
Le Guetteur	Détection de feu et de fumée.
Le Greffier	Détection de plaques d'immatriculation.
L'Arbitre	Détection de bagarre / comportement violent.
Le Silhouette	Segmentation (contour exact au pixel).
Le Secouriste	<i>Pas un modèle</i> : règle géométrique sur le squelette (torse à plat + boîte plus large que haute) pour repérer une chute.
Le Physionomiste	Reconnaissance faciale (MTCNN + InceptionResnetV1) avec mémoire persistante à deux niveaux (voir §3).

Isolation par caméra : chaque caméra possède son propre jeu des 9 modèles, chargé à la demande. Cela évite que le suivi temporel (`track_id`) d'une caméra ne se mélange avec celui d'une autre. La reconnaissance faciale, elle, reste **globale** : une personne reconnue l'est sur n'importe quelle caméra du réseau.

3 Reconnaissance faciale à deux mémoires

Pour éviter qu'un visage croisé une seule fois ne devienne immédiatement une "personne", le système utilise deux bases distinctes :



- **Seuil de confiance visage** : 92% minimum (évite les faux positifs sur des motifs/ombres).
- **Seuil de confirmation** : 5 apparitions similaires avant promotion.
- À la promotion, une **photo** est sauvegardée et l'interface demande un nom (ou permet de marquer "Inconnu N", numéroté automatiquement).
- Un nom peut être modifié à tout moment depuis `/bat` → *Gérer les personnes*.

Autorisations par point

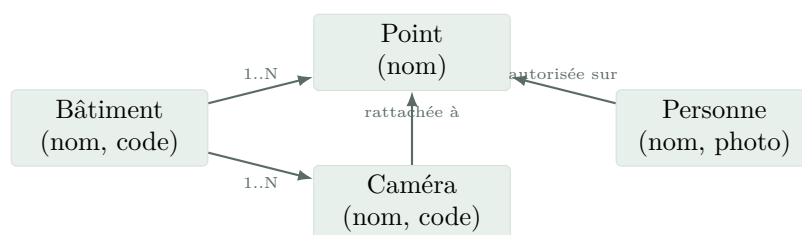
Chaque personne confirmée peut être **autorisée** sur une liste de points (ex : "Portail", "Bureau 3") propres à un bâtiment. Si elle est reconnue sur une caméra associée à un point où elle n'est **pas** autorisée, le système l'indique clairement (**présence non autorisée**) dans l'affichage et dans la narration. Une personne marquée "Inconnu" n'a par définition aucune autorisation nulle part.

4 Narration factuelle (sans IA générative)

Un point important de conception : la description en langage naturel de ce qui se passe ("*Détecté : 1 bus, 4 personnes.*") n'est **pas** générée par un grand modèle de langage. Une première version utilisait un LLM (Groq / Llama) pour un style plus "vivant", mais ce dernier inventait parfois des détails absents des données (noms, objets, actions). Le narrateur a donc été réécrit en **génération déterministe** : une phrase construite directement à partir des faits détectés, mot pour mot, sans aucune part de créativité — donc sans risque d'invention.

Le LLM (Groq, modèle `llama-3.1-8b-instant`) n'est utilisé que dans `/bat/chat`, où il répond à des questions libres **uniquement** à partir des données réelles fournies en contexte, avec instruction stricte de répondre "je ne sais pas" plutôt que d'inventer.

5 Modèle de données



Chaque **bâtiment** et chaque **caméra** possède un code unique généré automatiquement (ex : A1B2-C3D4), qui sert de moyen d'accès simple (pas de mot de passe) à `/cam` et `/bat`.

6 Stockage

Aucune base de données externe : tout est enregistré en fichiers JSON dans `db/`, pour rester simple et inspectable directement.

Fichier / dossier	Contenu
<code>buildings.json</code>	Bâtiments, points, caméras (avec leurs codes).
<code>faces_candidates.json</code>	Visages en observation (pas encore confirmés).
<code>faces_registry.json</code>	Personnes confirmées : nom, photo, autorisations.
<code>faces/</code>	Photos des personnes confirmées.
<code>cameras/<id>/detections_*.jsonl</code>	Historique des détections, par caméra et par jour.
<code>cameras/<id>/narration_*.jsonl</code>	Historique des phrases de narration, séparé des détections.

7 Sécurité

- Le serveur tourne en **HTTPS** (certificat auto-signé) : indispensable pour que les navigateurs mobiles autorisent l'accès à la caméra en dehors de `localhost`.
- Accès par **code unique** pour chaque caméra et chaque bâtiment — pas de compte utilisateur à ce stade (adapté à un usage interne sur réseau local).
- Les données biométriques (embeddings de visage) restent **stockées localement**, jamais transmises à un service externe.

8 Limites connues et pistes

Pour se rapprocher de la vision initiale

Par ordre de priorité si l'objectif est de revenir vers la promesse du dossier de pitch (comparatif en §1, "*Ce document par rapport au dossier de pitch*") :

1. **Détection de signaux précurseurs** (principe 2) : entraîner ou intégrer un modèle qui reconnaît un rôdage prolongé ou une posture d'agression à partir du squelette déjà extrait par Le Comportementaliste, plutôt que de se limiter à la chute (déjà survenue).
2. **Escalade par appel vocal** (principe 4) : brancher un service de téléphonie automatisée (ex. SIP + voix de synthèse) sur les alertes critiques déjà détectées (armes, feu, bagarre), avec confirmation requise du responsable.
3. **Reconnaissance faciale** (principe 1) : si l'écart reste problématique pour le jury ou pour des raisons de conformité, le module Physionomiste peut être désactivé sans toucher au reste de la chaîne — il est indépendant des 9 autres modèles.

Limites techniques du MVP actuel

- Les modèles communautaires (Sentinelle, Greffier, Arbitre) ont une précision plus faible que les modèles officiels YOLO11 — seuils de confiance relevés pour limiter les faux positifs, à surveiller en usage réel.
- La reconnaissance faciale peut créer plusieurs identifiants pour la même personne si l'éclairage/l'angle varie beaucoup (embeddings trop différents pour matcher) — fusion manuelle possible via suppression dans *Gérer les personnes*.

- Le narrateur factuel est fiable mais moins "vivant" qu'une génération libre — compromis assumé pour éliminer tout risque d'invention.
- Le chat (`/bat/chat`) dépend d'une clé API Groq externe — à surveiller en cas de changement de disponibilité du service.

YANFLÈ — TECH IMPACT — ORANGE SUMMER CHALLENGE 2026